

U.T. 3. Utilización de los objetos predefinidos del lenguaje.

Un objeto es una colección de propiedades (una serie de valores), de métodos (una serie de funciones) y de eventos (una serie de acciones). Para acceder a una propiedad o método de un objeto hay que concatenar ésta con el nombre del objeto por medio de un punto (.):

```
Objeto.propiedad;  
Objeto.metodo(argumentos_si_se_necesitan);
```

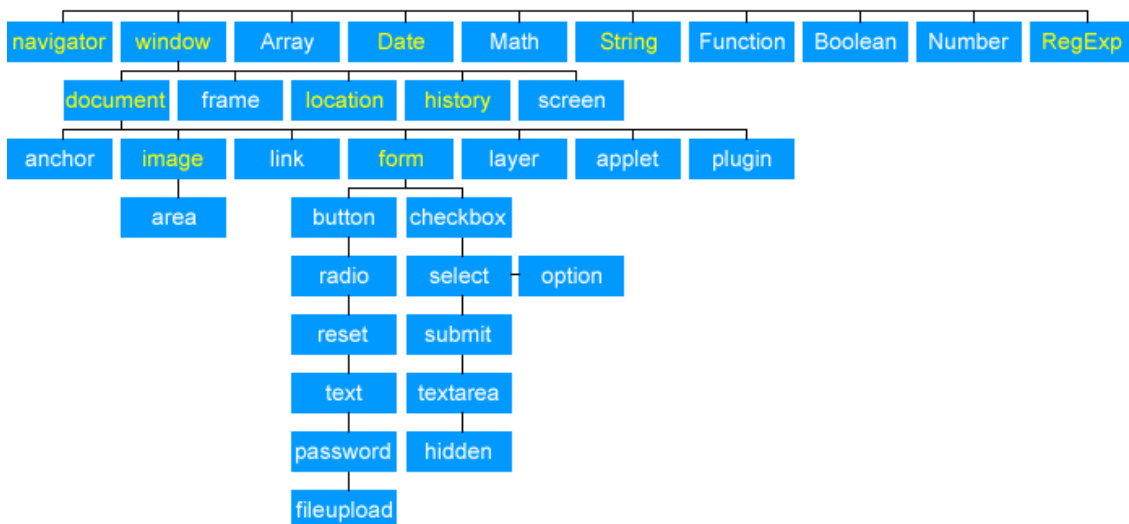


Figura 1. Jerarquía de objetos Javascript

2.1. Objetos nativos de JavaScript

Javascript proporciona una serie de objetos nativos que no dependen del navegador, por lo que podemos utilizarlos en cualquier momento. Como son Date, Math, Number y String. Hay más objetos nativos que no dependen del navegador que veremos en posteriores apartados. La sintaxis para crear un objeto es la siguiente:

```
var miObjeto = new Object();
```

2.1.1. El objeto Date

Para trabajar con fechas necesitamos instanciar un objeto de la clase Date y con él ya podemos realizar las operaciones que necesitemos.

Podemos crear el objeto con el día y hora actuales y por otro podemos crearlo con un día y hora distintos a los actuales. Esto depende de los parámetros que pasemos al construir los objetos.

Para crear un objeto fecha con el día y hora actuales colocamos los paréntesis vacíos al llamar al constructor de la clase Date.

Este objeto no tiene propiedades, únicamente tiene una serie de métodos con los que podremos trabajar.

Method	Description
getDate()	Returns the day of the month (from 1-31)
getDay()	Returns the day of the week (from 0-6)
getFullYear()	Returns the year (four digits)
getHours()	Returns the hour (from 0-23)
getMilliseconds()	Returns the milliseconds (from 0-999)
getMinutes()	Returns the minutes (from 0-59)
getMonth()	Returns the month (from 0-11)
getSeconds()	Returns the seconds (from 0-59)
getTime()	Returns the number of milliseconds since midnight Jan 1, 1970
getTimezoneOffset()	Returns the time difference between UTC time and local time, in minutes
getUTCDate()	Returns the day of the month, according to universal time (from 1-31)
getUTCDay()	Returns the day of the week, according to universal time (from 0-6)
getUTCFullYear()	Returns the year, according to universal time (four digits)
getUTCHours()	Returns the hour, according to universal time (from 0-23)
getUTCMilliseconds()	Returns the milliseconds, according to universal time (from 0-999)
getUTCMinutes()	Returns the minutes, according to universal time (from 0-59)
getUTCMonth()	Returns the month, according to universal time (from 0-11)
getUTCSeconds()	Returns the seconds, according to universal time (from 0-59)
getYear()	Deprecated. Use the <code>getFullYear()</code> method instead
parse()	Parses a date string and returns the number of milliseconds since midnight of January 1, 1970
setDate()	Sets the day of the month of a date object
setFullYear()	Sets the year (four digits) of a date object
setHours()	Sets the hour of a date object
setMilliseconds()	Sets the milliseconds of a date object
setMinutes()	Set the minutes of a date object
setMonth()	Sets the month of a date object
setSeconds()	Sets the seconds of a date object
setTime()	Sets a date and time by adding or subtracting a specified number of milliseconds to/from midnight January 1, 1970

setUTCDate()	Sets the day of the month of a date object, according to universal time
setUTCFullYear()	Sets the year of a date object, according to universal time (four digits)
setUTCHours()	Sets the hour of a date object, according to universal time
setUTCMilliseconds()	Sets the milliseconds of a date object, according to universal time
setUTCMinutes()	Set the minutes of a date object, according to universal time
setUTCMonth()	Sets the month of a date object, according to universal time
setUTCSeconds()	Set the seconds of a date object, according to universal time
setYear()	Deprecated. Use the setFullYear() method instead
toString()	Converts the date portion of a Date object into a readable string
toGMTString()	Deprecated. Use the toUTCString() method instead
toISOString()	Returns the date as a string, using the ISO standard
toJSON()	Returns the date as a string, formatted as a JSON date
toLocaleDateString()	Returns the date portion of a Date object as a string, using locale conventions
toLocaleTimeString()	Returns the time portion of a Date object as a string, using locale conventions
toLocaleString()	Converts a Date object to a string, using locale conventions
toString()	Converts a Date object to a string
toTimeString()	Converts the time portion of a Date object to a string
toUTCString()	Converts a Date object to a string, according to universal time
UTC()	Returns the number of milliseconds in a date string since midnight of January 1, 1970, according to universal time
valueOf()	Returns the primitive value of a Date object

```

<html>
<head>
<script type="text/javascript">
    function mostrarFechaHora()
    {
        var fecha;
        fecha=new Date();
        document.write('Hoy es ');
        document.write(fecha.getDate()+'/');
        document.write((fecha.getMonth()+1)+'/');
        document.write(fecha.getFullYear());
        document.write('<br>');
        document.write('Es la hora ');
        document.write(fecha.getHours()+' : ');
        document.write(fecha.getMinutes()+' : ');
        document.write(fecha.getSeconds());
    }
    mostrarFechaHora();
</script>

```

```
</head>
<body>
</body>
</html>
```

2.1.2. El objeto Math

El objeto Math permite realizar tareas matemáticas. Todas las propiedades / métodos del objeto Math pueden ser invocadas accediendo directamente al objeto Math sin necesidad de llamar al constructor.

Las propiedades del objeto Math son las siguientes:

Property	Description
E	Returns Euler's number (approx. 2.718)
LN2	Returns the natural logarithm of 2 (approx. 0.693)
LN10	Returns the natural logarithm of 10 (approx. 2.302)
LOG2E	Returns the base-2 logarithm of E (approx. 1.442)
LOG10E	Returns the base-10 logarithm of E (approx. 0.434)
PI	Returns PI (approx. 3.14)
SQRT1_2	Returns the square root of 1/2 (approx. 0.707)
SQRT2	Returns the square root of 2 (approx. 1.414)

Los métodos del objeto Math son los siguientes

Method	Description
abs(x)	Returns the absolute value of x
acos(x)	Returns the arccosine of x, in radians
asin(x)	Returns the arcsine of x, in radians
atan(x)	Returns the arctangent of x as a numeric value between -PI/2 and PI/2 radians
atan2(y,x)	Returns the arctangent of the quotient of its arguments
ceil(x)	Returns x, rounded upwards to the nearest integer
cos(x)	Returns the cosine of x (x is in radians)
exp(x)	Returns the value of E ^x
floor(x)	Returns x, rounded downwards to the nearest integer
log(x)	Returns the natural logarithm (base E) of x
max(x,y,z,...,n)	Returns the number with the highest value
min(x,y,z,...,n)	Returns the number with the lowest value
pow(x,y)	Returns the value of x to the power of y
random()	Returns a random number between 0 and 1
round(x)	Rounds x to the nearest integer
sin(x)	Returns the sine of x (x is in radians)
sqrt(x)	Returns the square root of x

tan(x)	Returns the tangent of an angle
---------------	---------------------------------

```
var num = 27.6;
document.write("El número es: " + num + "<br/>");
document.write("Math.floor()redondea hacia abajo: " + Math.floor(num)
+ "<br/>");
document.write("Math.ceil()redondea hacia arriba: " + Math.ceil(num) +
"<br/>");
document.write("Math.round()redondea hacia el entero más cercano: " +
Math.round(num));
document.write("Número aleatorio entre 0-1 =" + Math.random() +
"<br/>");
document.write("Número aleatorio entre 0-10 =" + Math.random() * 10 +
"<br/>");
document.write("Valor de PI =" + Math.PI + "<br/>");
document.write("Valor absoluto de -55 =" + Math.abs(-55) + "<br/>");
```

2.1.3. El objeto Number

No se suele utilizar ya que generalmente asignamos los valores numéricos directamente a una variable. Para utilizar la sintaxis sería la siguiente.

```
var num = new Number(value);
```

Si el parámetro pasado al constructor no se puede convertir, devolverá NaN.

Las propiedades del objeto Number son las siguientes

Property	Description
constructor	Returns the function that created the Number object's prototype
MAX_VALUE	Returns the largest number possible in JavaScript
MIN_VALUE	Returns the smallest number possible in JavaScript
NEGATIVE_INFINITY	Represents negative infinity (returned on overflow)

NaN	Represents a "Not-a-Number" value
POSITIVE_INFINITY	Represents infinity (returned on overflow)
prototype	Allows you to add properties and methods to an object

Los métodos del objeto Number son los siguientes

Method	Description
toExponential(x)	Converts a number into an exponential notation
toFixed(x)	Formats a number with x numbers of digits after the decimal point
toPrecision(x)	Formats a number to x length
toString()	Converts a Number object to a string
valueOf()	Returns the primitive value of a Number object

```
var count1, count2, txt;
count1 = new Number(6.257);
count2 = 6.258;
if (count1.toFixed(2) === count2.toFixed(2))
    txt = 'Estos valores son iguales en sus dos primeras posiciones decimales';
```

2.1.4. El objeto String

El objeto String va a permitir manipular cadenas de texto mediante su única propiedad y sus diferentes métodos.

Para crear objetos String lo podemos hacer de dos formas:

```
var txt = new String("string");
```

o:

```
var txt = "string";
```

Las propiedades del objeto String son las siguientes:

Property	Description
constructor	Returns the function that created the String object's prototype
length	Returns the length of a string
prototype	Allows you to add properties and methods to an object

Los métodos del objeto String son los siguientes:

Method	Description
charAt()	Returns the character at the specified index
charCodeAt()	Returns the Unicode of the character at the specified index
concat()	Joins two or more strings, and returns a copy of the joined strings
fromCharCode()	Converts Unicode values to characters
indexOf()	Returns the position of the first found occurrence of a specified value in a string
lastIndexOf()	Returns the position of the last found occurrence of a specified value in a string
match()	Searches for a match between a regular expression and a string, and returns the matches
replace()	Searches for a match between a substring (or regular expression) and a string, and replaces the matched substring with a new substring
search()	Searches for a match between a regular expression and a string, and returns the position of the match
slice()	Extracts a part of a string and returns a new string
split()	Splits a string into an array of substrings

substr()	Extracts the characters from a string, beginning at a specified start position, and through the specified number of character
substring()	Extracts the characters from a string, between two specified indices
toLowerCase()	Converts a string to lowercase letters
toUpperCase()	Converts a string to uppercase letters
trim()	Removes whitespace from both ends of a string
valueOf()	Returns the primitive value of a String object
anchor()	Creates an anchor
big()	Displays a string using a big font
blink()	Displays a blinking string
bold()	Displays a string in bold
fixed()	Displays a string using a fixed-pitch font
fontcolor()	Displays a string using a specified color
fontsize()	Displays a string using a specified size
italics()	Displays a string in italic
link()	Displays a string as a hyperlink
small()	Displays a string using a small font
strike()	Displays a string with a strikethrough
sub()	Displays a string as subscript text
sup()	Displays a string as superscript text

```

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-
1" />
<title>Palíndromos</title>
<script type="text/javascript">
function palindromo(cadena) {
    var resultado = "La cadena \""+cadena+"\" \n";
    var cadenaOriginal = cadena.toLowerCase();
    var letrasEspacios = cadenaOriginal.split("");
    var cadenaSinEspacios = "";
    // Se podría utilizar la funcion "replace" para obtener
    cadenaSinEspacios
    for(i in letrasEspacios) {
        if(letrasEspacios[i] != " ") {
            cadenaSinEspacios += letrasEspacios[i];
        }
    }
}

```

```
var letras = cadenaSinEspacios.split("");
var letrasReves = cadenaSinEspacios.split("").reverse();
var iguales = true;
for(i in letras) {
    if(letras[i] == letrasReves[i]) {
    }
    else {
        iguales = false;
    }
}
if(iguales) {
    resultado += " es un palíndromo";
}
else {
    resultado += " no es un palíndromo";
}
return resultado;
}
alert(palindromo("La ruta nos aporoto otro paso natural"));
alert(palindromo("Esta frase no se parece a ningun palindromo"));
</script>
</head>
<body>
</body>
</html>
```

2.2. Interacción de los objetos con el navegador

Javascript proporciona otra serie de objetos nativos que si dependen del navegador, y que nos permiten realizar acciones sobre él.

Las versiones 3.0 de los navegadores Internet Explorer y Netscape Navigator introdujeron el concepto de *Browser Object Model* o BOM, que permite acceder y modificar las propiedades de las ventanas del propio navegador.

Mediante BOM, es posible redimensionar y mover la ventana del navegador, modificar el texto que se muestra en la barra de estado y realizar muchas otras manipulaciones no relacionadas con el contenido de la página HTML.

El mayor inconveniente de BOM es que, al contrario de lo que sucede con DOM (Document Object Model), que veremos más adelante, ninguna entidad se encarga de estandarizarlo o definir unos mínimos de interoperabilidad entre navegadores.

Algunos de los elementos que forman el BOM son los siguientes:

- Crear, mover, redimensionar y cerrar ventanas de navegador.
- Obtener información sobre el propio navegador.
- Propiedades de la página actual y de la pantalla del usuario.
- Gestión de cookies.
- Objetos ActiveX en Internet Explorer.

El BOM está compuesto por varios objetos relacionados entre sí. El siguiente esquema muestra los objetos de BOM y su relación:

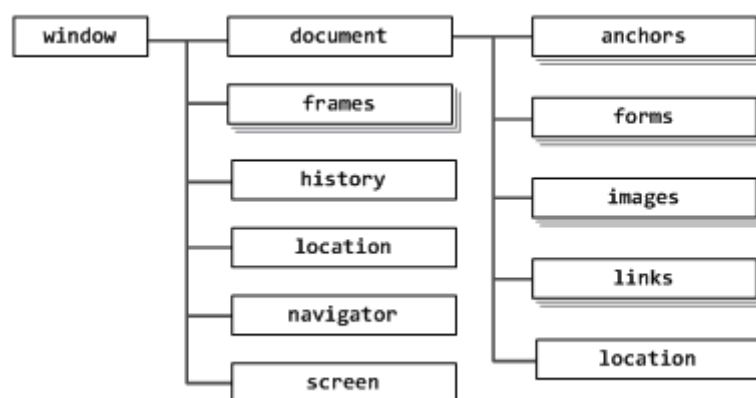


Figura 2. Jerarquía de objetos que forman el BOM

En el esquema anterior, los objetos mostrados con varios recuadros superpuestos son arrays. El resto de objetos, representados por un rectángulo

individual, son objetos simples. En cualquier caso, todos los objetos derivan del objeto window.

2.2.1. El objeto Navigator

Guarda información sobre el navegador que se está utilizando.

Las propiedades del objeto Navigator son:

Property	Description
appCodeName	Returns the code name of the browser
appName	Returns the name of the browser
appVersion	Returns the version information of the browser
cookieEnabled	Determines whether cookies are enabled in the browser
onLine	Boolean, returns <i>true</i> if the browser is on line, otherwise <i>false</i> .
platform	Returns for which platform the browser is compiled
userAgent	Returns the user-agent header sent by the browser to the server

Los métodos del objeto Navigator son:

Method	Description
javaEnabled()	Specifies whether or not the browser has Java enabled
taintEnabled()	Specifies whether or not the browser has data tainting enabled

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-
1" />
```

```

<title>El objeto Navigator</title>
</head>
<body>
<script type="text/javascript">
document.write("Valores de las propiedades del objeto Navigator\n");
document.write("<ul>\n");
for ( var prop in navigator ) {
    document.write("<li>"+prop+" "+navigator[prop]+" \n");
    if ( prop=="plugins" ) {
        for ( var i=0; i<navigator.plugins.length; i++ ) {
            document.write("Plug-in numero "+i+"<br><ul>\n");
            for ( var prop in navigator.plugins[i] ) {
                document.write("<li>"+prop+"
"+navigator.plugins[i][prop]+"</li>\n");
            }
            document.write("</ul><li>\n");
        }
    }
    if ( prop=="mimeTypes" ) {
        document.writeln( "<p>" );
        for ( var i=0; i<navigator.mimeTypes.length; i++ ) {
            document.write("Tipo MIME numero "+i+"<br><ul>\n");
            for ( var prop in navigator.mimeTypes[i] ) {
                document.write("<li>"+prop+"
"+navigator.mimeTypes[i][prop]+"</li>\n");
            }
            document.write("</ul>\n");
        }
    }
    document.write("</ul>\n");
}
</script>
</body>
</html>

```

2.2.2. El objeto Screen

Almacena información sobre la pantalla utilizada por el usuario, no contiene métodos, únicamente tiene seis propiedades que no podemos modificar, son de sólo lectura.

Las propiedades del objeto Screen son las siguientes:

Property	Description
availHeight	Returns the height of the screen (excluding the Windows Taskbar)
availWidth	Returns the width of the screen (excluding the Windows Taskbar)
colorDepth	Returns the bit depth of the color palette for displaying images
height	Returns the total height of the screen
pixelDepth	Returns the color resolution (in bits per pixel) of the screen
width	Returns the total width of the screen

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-
1" />
<title>El objeto Screen</title>
</head>
<body>
<script type="text/javascript">
var texte = "";
  with (screen)
  {
    texte += "\r\nwidth="      + width;
    texte += "\r\nheight="     + height;
    texte += "\r\navailWidth=" + availWidth;
    texte += "\r\navailHeight=" + availHeight;
    texte += "\r\ncolorDepth=" + colorDepth;
  }
  alert(texte);
</script>
```

```
</body>
</html>
```

2.2.3. El objeto Window

El objeto Window representa una ventana abierta en un navegador. Si un documento contiene marcos (<frame> o <iframe>), el navegador crea un objeto de la ventana para el documento HTML, y un objeto adicional de la ventana para cada marco.

No existe un estándar que se aplique al objeto Window, pero los principales navegadores lo soportan.

Las propiedades del objeto Window son las siguientes:

Property	Description
closed	Returns a Boolean value indicating whether a window has been closed or not
defaultStatus	Sets or returns the default text in the statusbar of a window
document	Returns the Document object for the window (See Document object)
frames	Returns an array of all the frames (including iframes) in the current window
history	Returns the History object for the window (See History object)
innerHeight	Sets or returns the inner height of a window's content area
innerWidth	Sets or returns the inner width of a window's content area
length	Returns the number of frames (including iframes) in a window
location	Returns the Location object for the window (See Location object)
name	Sets or returns the name of a window
navigator	Returns the Navigator object for the window (See Navigator object)
opener	Returns a reference to the window that created the window
outerHeight	Sets or returns the outer height of a window, including toolbars/scrollbars

outerWidth	Sets or returns the outer width of a window, including toolbars/scrollbars
pageXOffset	Returns the pixels the current document has been scrolled (horizontally) from the upper left corner of the window
pageYOffset	Returns the pixels the current document has been scrolled (vertically) from the upper left corner of the window
parent	Returns the parent window of the current window
screen	Returns the Screen object for the window (See Screen object)
screenLeft	Returns the x coordinate of the window relative to the screen
screenTop	Returns the y coordinate of the window relative to the screen
screenX	Returns the x coordinate of the window relative to the screen
screenY	Returns the y coordinate of the window relative to the screen
self	Returns the current window
status	Sets the text in the statusbar of a window
top	Returns the topmost browser window

Los métodos del objeto Window son los siguientes:

Method	Description
alert()	Displays an alert box with a message and an OK button
blur()	Removes focus from the current window
clearInterval()	Clears a timer set with setInterval()
clearTimeout()	Clears a timer set with setTimeout()
close()	Closes the current window
confirm()	Displays a dialog box with a message and an OK and a Cancel button
createPopup()	Creates a pop-up window
focus()	Sets focus to the current window

moveBy()	Moves a window relative to its current position
moveTo()	Moves a window to the specified position
open()	Opens a new browser window
print()	Prints the content of the current window
prompt()	Displays a dialog box that prompts the visitor for input
resizeBy()	Resizes the window by the specified pixels
resizeTo()	Resizes the window to the specified width and height
scroll()	
scrollBy()	Scrolls the content by the specified number of pixels
scrollTo()	Scrolls the content to the specified coordinates
setInterval()	Calls a function or evaluates an expression at specified intervals (in milliseconds)
setTimeout()	Calls a function or evaluates an expression after a specified number of milliseconds

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-
1" />
<title>El objeto Window</title>
<script type="text/javascript">
function Mover()
{
    Ventana.moveBy(5,5);
    i++;
    if (i<20)
        setTimeout('Mover()',100);
    else
        Ventana.close();
}
</script>
</head>
<body>
```

```

<script type="text/javascript">
    var dimensiones="left=100,top=100,width=100,height=100";
    var i=0;
    Ventana = window.open("", "", dimensiones);
    Ventana.document.write("Ventanas en movimiento");
    Ventana.moveTo(400,100);
    Mover();
</script>
</body>
</html>

```

2.2.4. El objeto Document

Se refiere a un documento HTML que se carga en la ventana del navegador. No confundir con Document de DOM.

Las propiedades del objeto Document son las siguientes:

Name	Description
alinkColor	A string that specifies the ALINK attribute.
anchors	An array containing an entry for each anchor in the document.
applets	An array containing an entry for each applet in the document.
bgColor	A string that specifies the BGCOLOR attribute.
classes	Creates a Style object that can specify the styles of HTML tags with a specific CLASS attribute.
cookie	Specifies a cookie.
domain	Specifies the domain name of the server that served a document.
embeds	An array containing an entry for each plug-in in the document.
fgColor	A string that specifies the TEXT attribute.
formName	A separate property for each named form in the document.
forms	An array a containing an entry for each form in the document.

height	The height of the document, in pixels.
ids	Creates a Style object that can specify the style of individual HTML tags.
images	An array containing an entry for each image in the document.
lastModified	A string that specifies the date the document was last modified.
layers	Array containing an entry for each layer within the document.
linkColor	A string that specifies the LINK attribute.
links	An array containing an entry for each link in the document.
plugins	An array containing an entry for each plug-in in the document.
referrer	A string that specifies the URL of the calling document.
tags	Creates a Style object that can specify the styles of HTML tags.
title	A string that specifies the contents of the TITLE tag.
URL	A string that specifies the complete URL of a document.
vlinkColor	A string that specifies the VLINK attribute.
width	The width of the document, in pixels.

Los métodos del objeto Document son los siguientes:

Name	Description
captureEvents	Sets the document to capture all events of the specified type.
close	Closes an output stream and forces data to display.
contextual	Uses contextual selection criteria to specify a Style object that can set the style of individual HTML tags.
getSelection	Returns a string containing the text of the current selection.
handleEvent	Invokes the handler for the specified event.
open	Opens a stream to collect the output of write or writeln methods.
releaseEvents	Sets the window or document to release captured events of the specified type, sending the event to objects further along the event hierarchy.

routeEvent	Passes a captured event along the normal event hierarchy.
write	Writes one or more HTML expressions to a document in the specified window.
writeln	Writes one or more HTML expressions to a document in the specified window and follows them with a newline character.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-
1" />
<title>El objeto Document</title>
</head>
<body>
<script type="text/javascript">
    document.write("Hola Mundo");
    document.bgColor = "red";
</script>
</body>
</html>
```

2.2.5. El objeto History

Contiene todas las direcciones que se han visitado en la sesión actual. Aunque no permite acceder a ellas, dispone de varios métodos para sustituir el documento actual por alguno que el usuario ya haya visitado.

Las propiedades del objeto History son las siguientes:

Property	Description
length	Read-only. Returns the number of elements in the session history, including the currently loaded page. For example, for a page loaded in a new tab this property returns 1.

current	Returns the URL of the active item of the session history. This property is not available to web content and is not supported by other browsers. Use <code>location.href</code> instead.
next	Returns the URL of the next item in the session history. This property is not available to web content and is not supported by other browsers.
previous	Returns the URL of the previous item in the session history. This property is not available to web content and is not supported by other browsers.

Los métodos del objeto History son los siguientes:

Method	Description
history.back()	Goes to the previous page in session history, the same action as when the user clicks the browser's Back button. Equivalent to <code>history.go(-1)</code>. Note: Calling this method to go back beyond the first page in the session history has no effect and doesn't raise an exception.
history.forward()	Goes to the next page in session history, the same action as when the user clicks the browser's Forward button; this is equivalent to <code>history.go(1)</code>. Note: Calling this method to go back beyond the last page in the session history has no effect and doesn't raise an exception.
history.go(<i>integerDelta</i>)	Loads a page from the session history, identified by its relative location to the current page, for example -1 for the previous page or 1 for the next page. When <i>integerDelta</i> is out of bounds (e.g. -1 when there are no previously visited pages in the session history), the method doesn't do anything and doesn't raise an exception. Calling <code>go()</code> without parameters or with a non-integer argument has no effect (unlike Internet Explorer, which supports string URLs as the argument).

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
```

```
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>El objeto History</title>
</head>
<body>
<script type="text/javascript">
    document.write(history.length);
</script>
</body>
</html>
```

2.2.6. El objeto Location

Representa la URL de la página HTML que se muestra en la ventana del navegador y proporciona varias propiedades y métodos útiles para el manejo de la URL.

Las propiedades del objeto Location son las siguientes:

Property	Description
hash	Returns the anchor portion of a URL
host	Returns the hostname and port of a URL
<u>hostname</u>	Returns the hostname of a URL
<u>href</u>	Returns the entire URL
<u>pathname</u>	Returns the path name of a URL
<u>port</u>	Returns the port number the server uses for a URL
<u>protocol</u>	Returns the protocol of a URL
<u>search</u>	Returns the query portion of a URL

Los métodos del objeto Location son los siguientes:

Method	Description
<u>assign()</u>	Loads a new document
<u>reload()</u>	Reloads the current document
<u>replace()</u>	Replaces the current document with a new one

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-
1" />
<title>El objeto Location</title>
</head>
<body>
<script type="text/javascript">
    location.href = "http://www.google.es";
</script>
</body>
</html>
```

2.3. Generación de elementos HTML desde código JavaScript

De forma sencilla mediante el método `document.write()` podemos crear y manipular todo tipo de objetos HTML.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-
1" />
<title>Código HTML desde JavaScript</title>
</head>
<body>
<script type="text/javascript">
    document.write("<form name=\"cambiacolor\">");
```

```

        document.write("<b>Selecciona un color para el fondo de
página:</b><br>");
        document.write("<select name=\"color\">");
        document.write("<option value=\"red\">Rojo</option>");
        document.write("<option value=\"blue\">Azul</option>");
        document.write("<option value=\"yellow\">Amarillo</option>");
        document.write("<option value=\"green\">Verde</option>");
        document.write("<option value=\"black\">Negro</option>");
        document.write("<option value=\"white\">Blanco</option>");
        document.write("</select>");
        document.write("<input type=\"button\" value=\"Modifica el
color\"
                        onclick=\"document.bgColor
                        =
document.cambiacolor.color.value\">");
        document.write("</form>");
</script>
</body>
</html>

```

Veremos más adelante como hacer lo mismo con el árbol DOM.

2.4. Aplicaciones prácticas de los marcos

Los marcos o frames sirven para distribuir mejor los datos de las páginas, ya que permiten mantener fijas algunas partes, como pueden ser el logotipo y la barra de navegación, mientras que otras sí pueden cambiar. Además de mejorar la funcionalidad, pueden mejorar también la apariencia.

Cada uno de los marcos de una página, contiene un documento HTML individual. Por ejemplo, en la imagen de la derecha puedes ver una página con dos marcos. El marco izquierdo contiene el documento **menu.htm** y el derecho el documento **perros.htm**. Para poder visualizar la página de este modo, hemos tenido que abrir en el navegador el documento **marcos.htm**, que es el que en este caso contiene el grupo de marcos.

Si queremos utilizar marcos debemos crear una página con la definición del conjunto de marcos.

Los conjuntos de marcos se definen a través de las etiquetas `<frameset>` y `</frameset>`, que van después de la etiqueta `<head>`. A través de estas

etiquetas se indica el número de marcos en que se dividirá la ventana, cada uno de los cuales será una especie de subventana.

Cuando se insertan las etiquetas `<frameset>` y `</frameset>` no hay que insertar las etiquetas `<body>` y `</body>`, ya que el cuerpo del documento será el cuerpo de las páginas que se carguen en cada uno de los marcos del conjunto de marcos.

Los atributos que pueden especificarse sobre la etiqueta `<frameset>` son los siguientes:

Atributo	Significado
cols	tamaño de cada una de las columnas en que se divide el documento
rows	tamaño de cada una de las columnas en que se divide el documento
frameborder	aparece o no el borde de los marcos
framespacing	separación entre los marcos
border	grosor del borde
bordercolor	color del borde

También es posible incluir asteriscos como valores para los atributos `cols` y `rows`. Este valor indica que dicha fila o columna ocupará todo lo que quede de ventana o subventana. Cuando existan varias columnas o filas con este valor, se repartirán de forma equitativa lo que quede de ventana.

Para indicar qué documento se cargará en cada uno de los marcos habrá que insertar la etiqueta `<frame>` por cada uno de los marcos de la ventana. Estas etiquetas estarán entre las etiquetas `<frameset>` y `</frameset>`, y no necesitan etiqueta de cierre.

Es posible modificar los siguientes atributos de un marco:

Atributo	Significado
frameborder	aparece o no el borde del marco
name	nombre del marco
noresize	si aparece, el usuario no podrá redimensionar el tamaño de este marco

marginwidth	anchura del margen con respecto a los bordes del marco
marginheight	altura del margen con respecto a los bordes del marco
scrolling	se mostrará o no la barra de desplazamiento cuando la página del marco no se pueda visualizar completamente en él
src	documento que se cargará en el marco

Siempre que creamos una página queremos que pueda ser visitada por el mayor número de usuarios, por lo que el hecho de que hayan usuarios cuyos navegadores no soportan los marcos puede resultar un problema.

Las etiquetas `<noframes>` y `</noframes>` se utilizan para que su contenido sea visualizado en aquellos navegadores que no soportan los marcos.

Si un navegador no soporta los marcos, no cargará ningún documento en ningún marco, por lo que tendremos que mostrar el cuerpo del documento actual de algún modo. Por ello, han de introducirse las etiquetas `<body>` y `</body>` entre las etiquetas `<noframes>` y `</noframes>`.

index.html

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-
1" />
<title>Frameset</title>
</head>
<frameset cols="25%,*">
  <frame src="frame1.html">
  <frame src="frame2.html">
</frameset>
</html>
```

frame1.html

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
```

```
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>Frame 1</title>
</head>
<body style="background-color:#8F8FBD">
<h3>Frame 1</h3>
</body>
</html>
```

frame2.html

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>Frame 2</title>
</head>
<body style="background-color:#EBC79E">
<h3>Frame 2</h3>
</body>
</html>
```

2.4.1. Uso de marcos con JavaScript

Mediante JavaScript podemos tomar el control de los marcos de una página. Lo haremos utilizando las propiedades frames, parent y top del objeto Window. La propiedad frames contiene una lista con los marcos de la página. La propiedad parent accede al objeto que contiene los marcos. La propiedad top corresponde a la ventana de nivel superior.

index.html

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
```

```
<title>Frameset</title>
</head>
<frameset cols="25%,*">
  <frame src="frame1.html">
  <frame src="frame2.html">
</frameset>
</html>
```

frame1.html

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-
1" />
<title>Frame 1</title>
</head>
<body>
<h3>Frame 1</h3>
<form>
<select name="color">
<option value="red">Rojo
<option value="green">Verde
<option value="Blue">Azul
</select>
<select name="marcos">
<option value="0">Izquierda
<option value="1">Derecha
</select>
</form>
</body>
</html>
```

frame2.html

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-
1" />
<title>Frame 2</title>
```

```

</head>
<body>
<h3>Frame 2</h3>
<script type="text/javascript">
function cambiarColor(){
    campoColor = parent.frames[0].document.forms[0].color;
    colorin = campoColor.options[campoColor.selectedIndex].value;
    campoFrame = parent.frames[0].document.forms[0].marcos;
    idFrame = campoFrame.options[campoFrame.selectedIndex].value;
    window.parent.frames[idFrame].document.bgColor = colorin;
}
</script>
<form>
<input type="Button" value="Cambiar Color" onclick="cambiarColor()">
</form>
</body>
</html>

```

2.5. Gestión de las ventanas

Veremos cómo abrir y cerrar ventanas, cómo controlar la apariencia de las ventanas y cómo gestionar la comunicación entre varias ventanas.

2.5.1. Abrir y cerrar nuevas ventanas

Las ventanas poseen diferentes propiedades y métodos, uno de los métodos es `open()` que creará una nueva ventana, cuenta con tres parámetros:

- URL de la página web que albergará la nueva ventana
- Nombre asignado a la nueva ventana
- Atributos que definen la apariencia de la nueva ventana

Otro de los métodos es `close()` que cerrará la ventana sobre la que se invoque.

```

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">

```

```

<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>Abrir Cerrar Ventanas</title>
<script type="text/javascript">
    function open_win(){
        nuevaVentana = window.open("http://www.google.es");
    }
    function close_win(){
        nuevaVentana.close();
    }
</script>
</head>
<body>
<form>
    <input type="button" value="Open Window" onclick="open_win()">
    <input type="button" value="Close Window" onclick="close_win()">
</form>
</body>
</html>

```

2.5.2. Apariencia de las ventanas

Podemos también controlar la apariencia de las ventanas modificando sus propiedades con JavaScript, podemos decidir tamaño, ubicación y elementos que contendrá la ventana.

Propiedades que nos permitirán controlar la apariencia de las ventanas

Width	Ajusta el ancho de la ventana. En pixels
Height	Ajusta el alto de la ventana
Top	Indica la posición de la ventana. En concreto es la distancia en pixels que existe entre el borde superior de la pantalla y el borde superior de la ventana.
Left	Indica la posición de la ventana. En concreto es la

	distancia en pixels que existe entre el borde izquierdo de la pantalla y el borde izquierdo de la ventana.
Scrollbars	Para definir de forma exacta si salen o no las barras de desplazamiento. scrollbars=NO hace que nunca salgan. Scrollbars=YES hace que salgan (siempre en ie y solo si son necesarias en NTS).
Resizable	Establece si se puede o no modificar el tamaño de la ventana. Con resizable=YES se puede modificar el tamaño y con resizable=NO se consigue un tamaño fijo.
Directories (barra directorios)	A partir de aquí se enumeran otra serie de propiedades que sirven para mostrar o no un elemento de la barra de navegación que tienen los navegadores más populares, como podría ser la barra de menús o la barra de estado. Cuando ponemos el atributo=YES estamos forzando a que ese elemento se vea. Cuando ponemos atributo=NO lo que hacemos es evitar que ese elemento se vea.
Location (barra direcciones)	
Menubar (barra de menús)	
Status (barra de estado)	
Titlebar (la barra del título)	
Toolbar (barra de herramientas)	

2.5.3. Comunicación entre ventanas

La comunicación podrá ser bidireccional:

1. Desde la ventana principal a la secundaria.
2. Desde la ventana secundaria a la principal. (Desde la secundaria por motivos de seguridad los navegadores no permiten realizar ciertas operaciones)

Si queremos comunicar desde la ventana principal hacia la secundaria necesitamos disponer de una referencia de dicha ventana secundaria. La referencia será el nombre que le pongamos a la ventana secundaria.

Si la comunicación es desde la ventana secundaria a la principal debemos utilizar el atributo opener de dicha ventana secundaria, que referencia a la ventana principal.

Cuando abrimos una ventana utilizando el método open del objeto window asignamos un nombre a la ventana para referirnos a ella utilizando HTML. Pero si queremos referirnos a ella utilizando Javascript necesitaremos utilizar otro nombre.

La referencia Javascript a la ventana que se acaba de abrir se obtiene gracias al valor de retorno del método open. Para guardar la referencia asignamos el valor de retorno del método a una variable. A partir de ese momento la variable será un sinónimo del objeto window, es decir, será como si fuera el objeto window del popup y por lo tanto podremos acceder a los métodos y propiedades de la ventana secundaria a partir de esa variable.

```
referenciaVentana = window.open (mi_url.html , nombre , width=100,  
height=300);
```

Luego podremos acceder a los métodos y propiedades de esta manera.

```
referenciaVentana.close();  
referenciaVentana.document.backgroundColor = red;
```